Tim Book

An Intro to Docker

GENERAL ASSEMBLY

Agenda

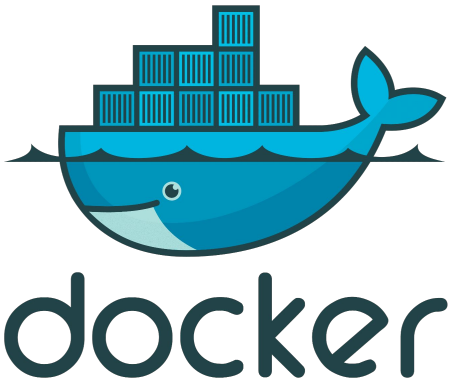
- What is Docker and how does it work?
- Who uses Docker?
- Anatomy of a Docker image and some Docker vocabulary
- Building and running Docker images and containers
- Cleaning up Docker images
- **Special Demo:** Running Jupyter Notebooks locally on the GPU using a Docker container

 © 2023 General Assembly

What is Docker?

Docker is an OS-level **virtualization software** for delivering portable software in packages called **containers**.

That's a mouthful. Let's talk about what that means.



What is virtualization?

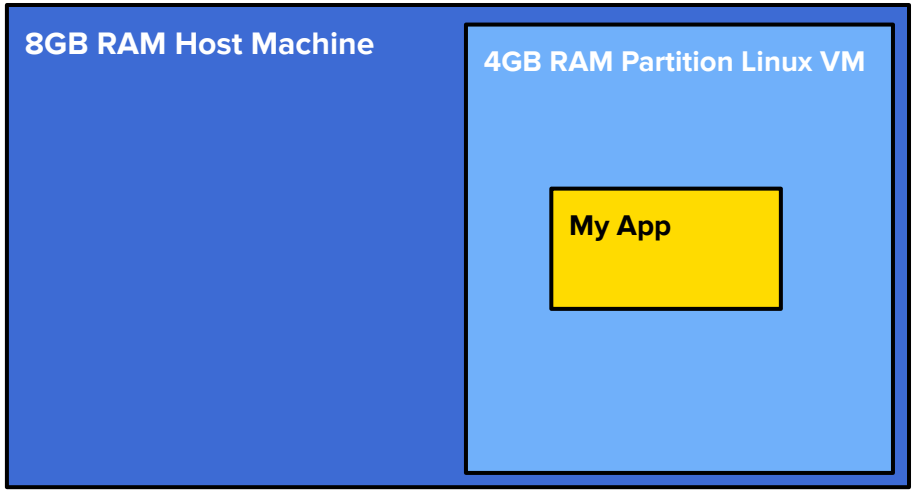
Virtualization is the act of creating virtual instances of another operating system on top of a host operating system.

Docker is for virtualization, but it does ***NOT*** create a virtual machine (VM)!

 © 2023 General Assembly

Virtual Machines

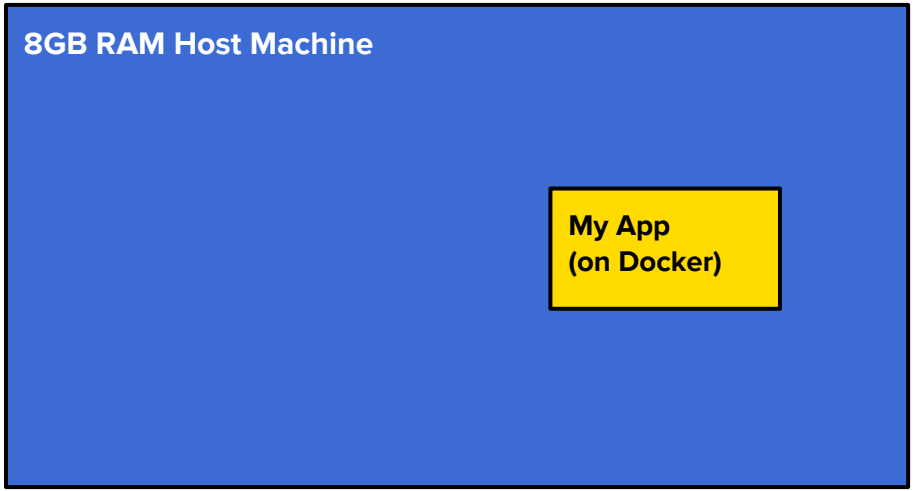
It is easy to create a Linux VM using a **hypervisor** such as Oracle VirtualBox. VMs divide system resources between two operating systems.



Docker containers share system resources

Instead of partitioning your RAM like in a VM, Docker containers use system resources as if they were processes on your machine!

This is why Docker is considered to be much more **lightweight** than a VM. Containers can be created and destroyed much more easily than VMs.



 © 2023 General Assembly

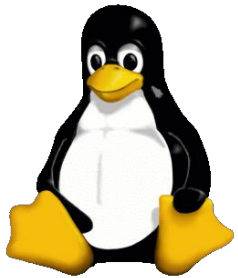
Linux Notes

Docker is for Linux only*!

Because Docker shares memory resources with its host machine, the Docker container and the host OS need to be the same. Almost all Docker containers are **Linux**.

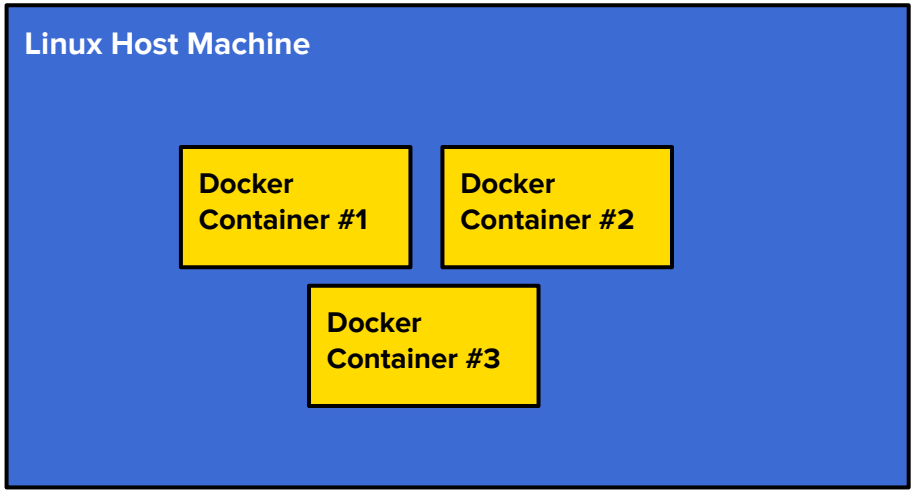
It is (currently) not possible to create a Mac OS Docker container.

*It is possible (but rare) to create a Windows Docker container (mostly for Microsoft SQL servers).



So how do we get around this?

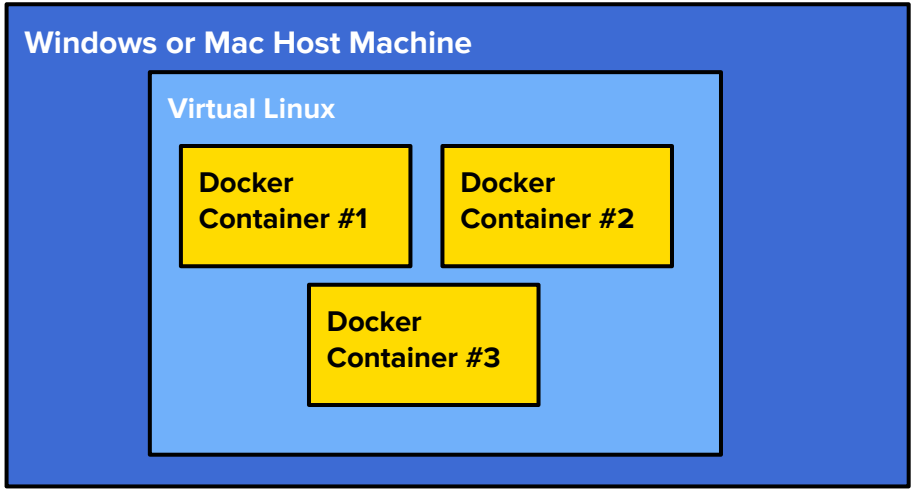
Well, if you're on Linux like me, you're already good to go!



 © 2023 General Assembly

So how do we get around this?

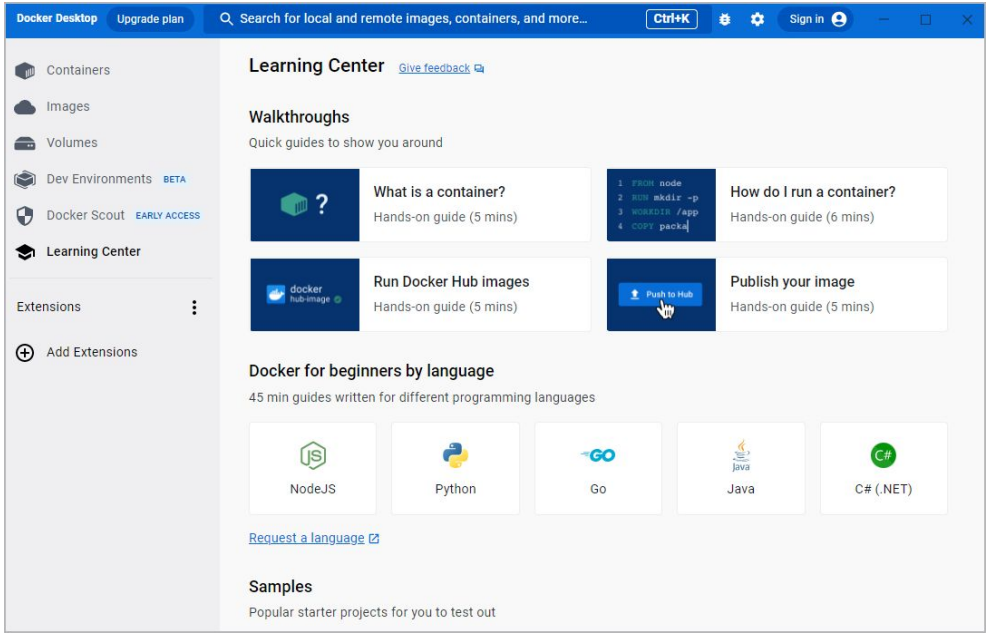
On Mac and Windows, Docker will need to virtualize a whole Linux kernel first, then put your containers on top of it. **(You won't notice this happening.)**



Docker Desktop

For MacOS and Windows, you'll use **Docker Desktop** to emulate a Linux VM in order to use Docker. Docker Desktop is actually very cool, it has GUI helper tools for you!

You must also keep this program running for Docker to work from the CL!



 © 2023 General Assembly

Installation

Linux: You know what to do for your personal distro!

MacOS: Install Docker Desktop here.

Windows: Install Docker Desktop here.

Who uses Docker?

We can divide Docker users into two camps

Camp #1:

“I’m going to containerize my web app. The app container will be able to communicate with my SQL database in a separate container. My containers talk to each other. **My containers are meant to stay running for months at a time and occasionally undergo maintenance.**”

These are DevOps people and web developers. Not us!

 © 2023 General Assembly

We can divide Docker users into two camps

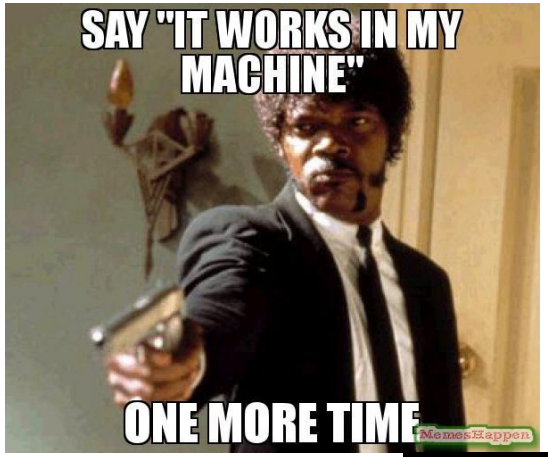
Camp #2:

“I want my program to run on this exact set of library versions no matter who runs it. I want total control over the environment so no one can say it doesn’t work on their machine. **My containers are meant to do one job and then shut down immediately.**”

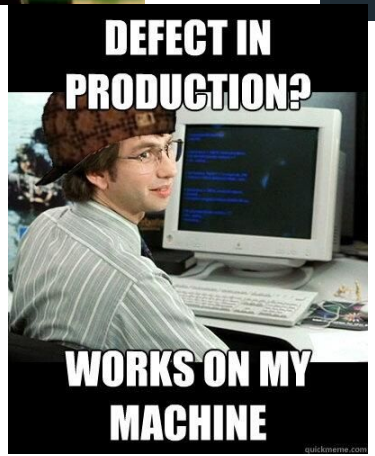
This is us!!!

Know Your Meme: “It works on my machine!”









Today's specific use-case

- TensorFlow-GPU can only run on NVIDIA GPUs
- It's really difficult to get TensorFlow-GPU running on Windows
- TensorFlow docs literally tell you the best way to get around all this confusion is to use Docker.
- They also make it really easy to use Docker for TensorFlow by making a **tensorflow-gpu** Docker image readily available to us!

Docker Anatomy and Vocab



Docker Vocab

- A **Docker image** is the read-only template from which new Docker containers can be created. You can create a new Docker image with:
  - `docker build -t [stuff]`
- To make a new Docker image, you base them off of pre-existing older Docker images. These images are created by other users or orgs and are hosted on **DockerHub**.
- Additional settings for your new Docker image are specified in a special file called **Dockerfile**.

 © 2023 General Assembly

Docker Vocab

- A **Docker container** is a running program that is the execution of a Docker image. For us, Docker containers should only live for as long as code is running. You can run a container with:
  - `docker run [stuff]`
- A **Docker cluster** is a group of multiple Docker containers running simultaneously, often talking to each other. These can be orchestrated via running Docker in **Swarm mode** or via a separate program called **Kubernetes**.

Docker Vocab: Daemons!

- A **daemon** (pronounced like “demon”) is a term that refers to a background process running on your machine.
- The **Docker daemon** is a daemon that is waiting for your Docker commands.
    - On Linux, it is a background process you can start and stop from the CL
    - On Mac/Windows, the daemon is running as long as you have **Docker Desktop** running. (Docker Desktop is what simulates a Linux VM upon which you run Linux containers).

 © 2023 General Assembly

docker run

Hello World!



Step 1: Acquire the Docker image

We're going to pull our pre-built Docker image directly from DockerHub. This will be the classical tutorial image: Hello World!

```
docker pull hello-world
```

The hello-world image is now downloaded to your machine. You only need to do this once (unless you want to pull an updated version later). You can view all of your images with:

```
docker image ls
```

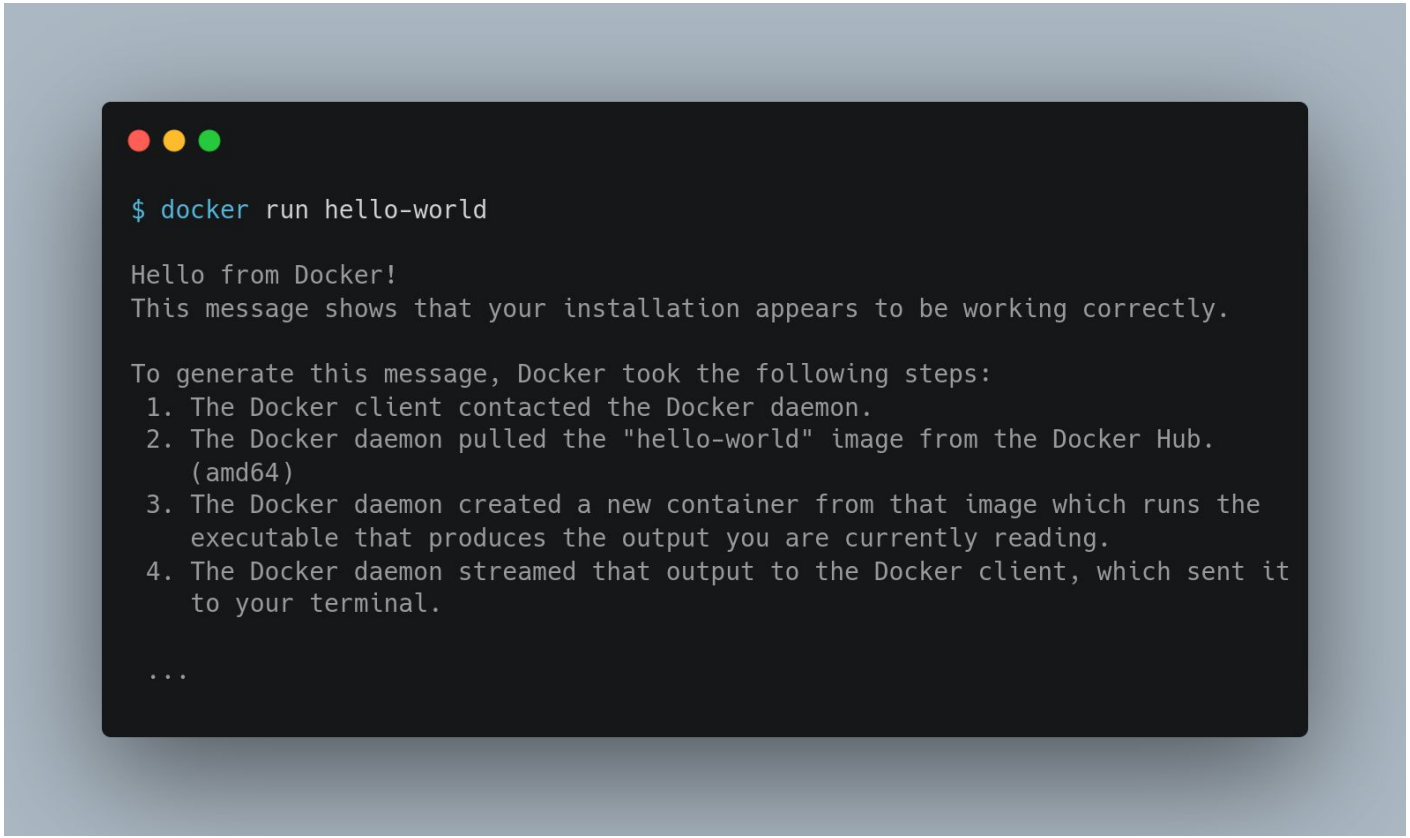
 © 2023 General Assembly

Step 2: Run the image as a container

Now that we have the image, we can run it! This is as easy as:

```
docker run hello-world
```

The hello-world container does one thing: Runs a program that prints some basic information to the screen. It's mostly used to confirm you have Docker installed properly.


$ docker run hello-world
Hello from Docker!
This message shows that your installation appears to be working correctly.
To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the
executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
to your terminal.
...

Something harder

The output to hello-world suggests we run a slightly more complicated command as a test:

```
docker run -it ubuntu bash
```

Something harder

The output to hello-world suggests we run a slightly more complicated command as a test:

```
docker run -it ubuntu bash
```



Running an image without pulling it first is fine! Docker will pull it for you automatically.

 © 2023 General Assembly

Something harder

The output to hello-world suggests we run a slightly more complicated command as a test:

```
docker run -it ubuntu bash
```



Running an image without pulling it first is fine! Docker will pull it for you automatically.



image name

 © 2023 General Assembly

Something harder

The output to hello-world suggests we run a slightly more complicated command as a test:

```
docker run -it ubuntu bash
```

Running an image without pulling it first is fine! Docker will pull it for you automatically.

image name

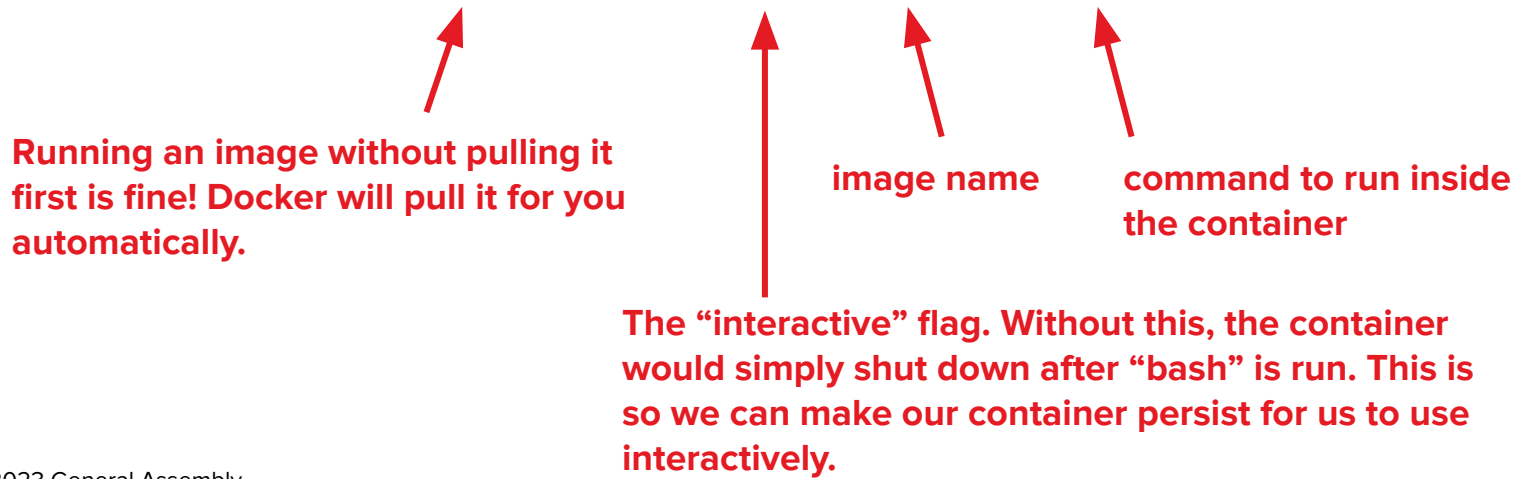
command to run inside the container

 © 2023 General Assembly

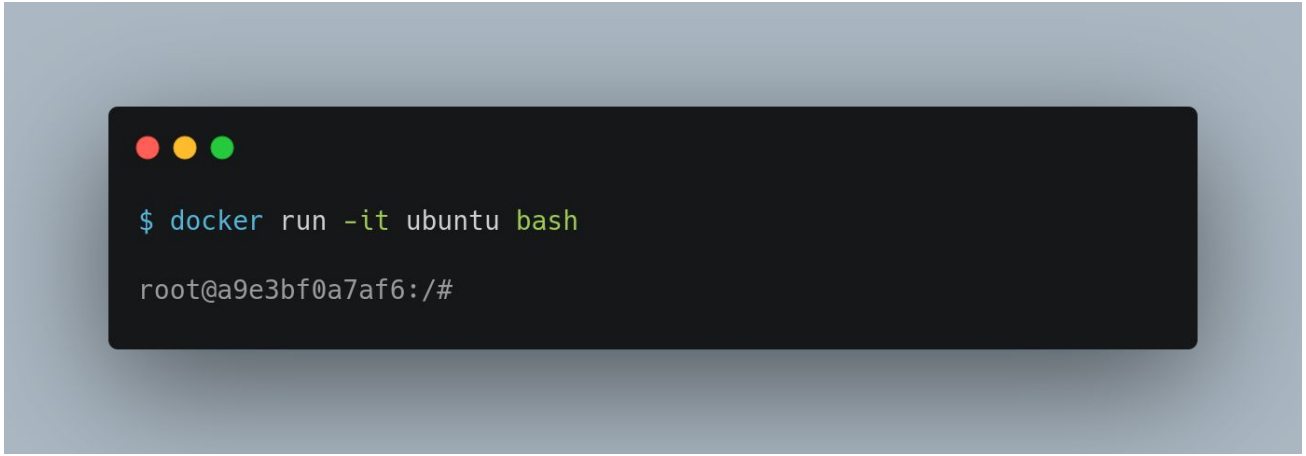
Something harder

The output to hello-world suggests we run a slightly more complicated command as a test:

```
docker run -it ubuntu bash
```



 © 2023 General Assembly



 © 2023 General Assembly

Building Your Own Images

How to build an image

In this demo I'll

1. Write the code I want executed in a Docker container
2. Build a Dockerfile to orchestrate how I want my container to run
3. Build the Docker image
4. Run a Docker container

Step 1: Write code!

Our goal will be to writer a Docker image that generates a random number for us. To start things off, let's write a simple Python script:

random-gen.py

```
import sys
import numpy as np

print(sys.version)

num = np.random.randint(1, 100)
print(f"Random number: {num}")
```

 © 2023 General Assembly

Step 2: Create a Dockerfile

The **Dockerfile** is what determines how **Docker build** will work. It's a list of commands that are run in order. We'll go over them individually:

Dockerfile

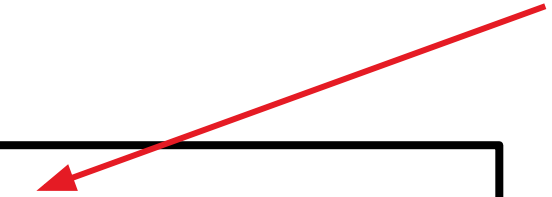
```
FROM python:3.9
WORKDIR /app
RUN pip install numpy
COPY random-gen.py .
CMD ["python", "random-gen.py"]
```

 © 2023 General Assembly

Step 2: Create a Dockerfile

Dockerfile

```
FROM python:3.9
WORKDIR /app
RUN pip install numpy
COPY random-gen.py .
CMD ["python", "random-gen.py"]
```

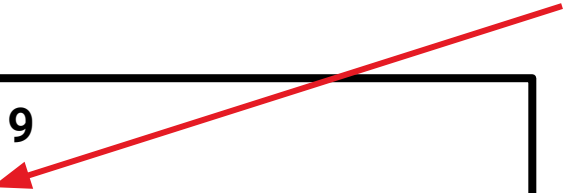


“FROM” specifies which image we are basing our new image on. 3.9 is the release name. Think of it like a version number.

Step 2: Create a Dockerfile

Dockerfile

```
FROM python:3.9
WORKDIR /app
RUN pip install numpy
COPY random-gen.py .
CMD ["python", "random-gen.py"]
```



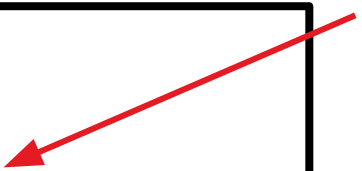
“WORKDIR” sets our working directory - the directory from which all relative paths will refer to. This is in the file tree of the Docker container, not the host machine. If the folder doesn’t exist yet, it will be created.

 © 2023 General Assembly

Step 2: Create a Dockerfile

Dockerfile

```
FROM python:3.9
WORKDIR /app
RUN pip install numpy
COPY random-gen.py .
CMD ["python", "random-gen.py"]
```



"RUN" is the command that will run shell commands on the Docker container. You generally use this to set up your environment. You cannot use "RUN" to print things to the screen, unfortunately.

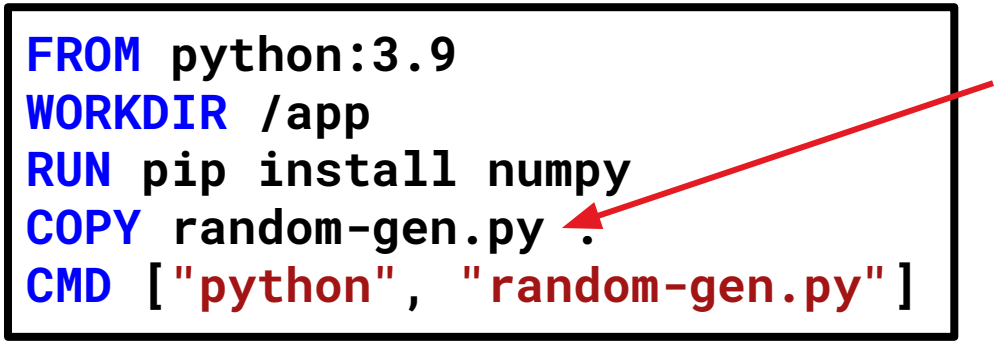
TIP: Docker actually layers Docker images every time you use a "RUN" command. So it's faster to bundle multiple command line commands into one step. For example, you can use the "command1 && command2" trick to combine steps.

Rule of thumb is: the fewer "RUN"s, the better.

Step 2: Create a Dockerfile

Dockerfile

```
FROM python:3.9
WORKDIR /app
RUN pip install numpy
COPY random-gen.py .
CMD ["python", "random-gen.py"]
```



“COPY” is one way to bring files from your local machine onto your Docker container. In our example, we’ll run our “build” command in the same directory as random.py. We will move it to the current directory on our container (.). In this example, it’s our working directory /app.

Step 2: Create a Dockerfile

Dockerfile

```
FROM python:3.9
WORKDIR /app
RUN pip install numpy
COPY random-gen.py .
CMD ["python", "random-gen.py"]
```

Dockerfiles may only have one “CMD” command, and only at the end. This is the one piece of command line code that is run at the end. It’s in the format of a list of strings, where each is combined to form the overall command. This one runs “python random-gen.py”

Step 3: Build the Docker Image

We've now done two things:

1. We've written Python code we wish to execute on a Docker container
2. We've written the Dockerfile that sets up our future container and orchestrates what will happen when it's executed

The next step is to **build** the Dockerfile and accompanying code into a **Docker image**.

 © 2023 General Assembly

Step 3: Build the Docker Image

We can build a Docker image from a Dockerfile with:

```
docker build -t image-name .
```

 © 2023 General Assembly

Step 3: Build the Docker Image

We can build a Docker image from a Dockerfile with:

```
docker build -t image-name .
```



The Dockerfile is here, in this folder.



Your Docker image name here. Usually this is the name of the application you're building.

Step 3: Build the Docker Image

So let's do it!

```
docker build -t rng .
```

Once it's done, you can verify your image has been built with:

```
docker image ls
```

 © 2023 General Assembly

Step 4: Run the Container!

Finally! We can run our Docker container with:

```
docker run rng
```

Note the output! You're running Python code on a different version than your host machine! You're doing the Docker thing!!!

Cleanup Step



Cleaning it all up

When you ran `docker image ls` you might have noticed how large Docker images can be. Some frequently used images out there can be several gigabytes. If you're not paying attention, these can really stack up on your computer.

Let's learn how to delete them.

 © 2023 General Assembly

Cleaning it all up

First, let's see what we see on my machine:



 © 2023 General Assembly

Cleaning it all up



To remove an image, the general code for this is:

```
docker rmi [some-identifier]
```

BUT THIS WON'T WORK YET!

Cleaning it all up

If you tried this, Docker would tell you it can't delete the image because a container in use is based on it. Even if your code has finished, this will be true!

You can check out all "running" Docker containers with:

```
docker ps -a
```

You'll see your history of run containers. Even "exited" containers are still seen as "in use" by Docker! We can clear this with a quick:

```
docker container prune
```

 © 2023 General Assembly

Cleaning it all up

After you've pruned these dangling containers, you can run

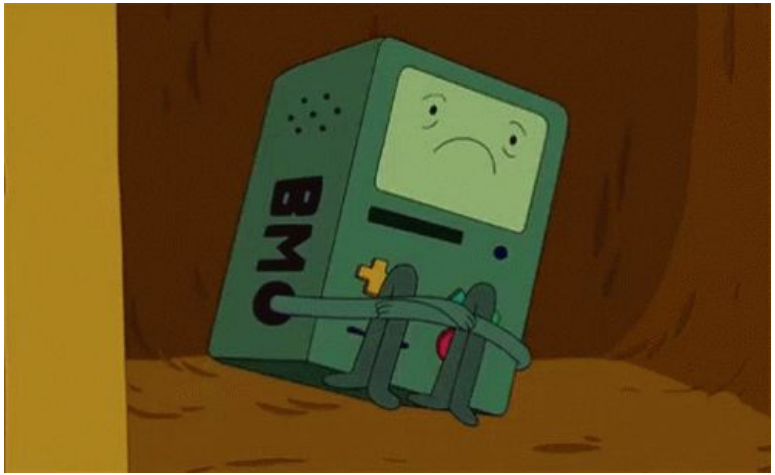
`docker rmi rng` or **`docker rmi 9beb`**



 © 2023 General Assembly

Special Demo: Keras on the GPU ...Locally!

Obvious Warning



This demo demonstrates the preferred way to fit TensorFlow models on your GPU.

I, your instructor, have a dedicated GPU in my home-built Desktop. You probably don't.

Additionally, none of this will work on a Mac, even if you have a GPU or fancy M1 chip!

You probably can't follow along with this. That's ok! One day you may want to do this sort of thing on a cloud instance. Same skills!

TensorFlow Docs

TensorFlow provides excellent documentation on how to run Keras on Docker. We won't even need to create our own image or Dockerfile!

Check them out here.

Our Docker Command

(run from inside this lesson repo!)

```
docker run \
    –gpus all \
    -v $(pwd):/home \
    tensorflow/tensorflow:latest-gpu-py3 \
    /bin/bash -c “pip3 install scikit-learn && python3
/home/keras-on-docker.py”
```

 © 2023 General Assembly

Our Docker Command

(run from inside this lesson repo!)

```
docker run \
    --gpus all \
    -v $(pwd):/home \
    tensorflow/tensorflow:latest-gpu-py3 \
    /bin/bash -c “pip3 install scikit-learn && python3
/home/keras-on-docker.py”
```



Run on all available detectable GPUs

Our Docker Command

(run from inside this lesson repo!)

```
docker run \
    –gpus all \
    -v $(pwd):/home \
    tensorflow/tensorflow:latest-gpu-py3 \
    /bin/bash -c “pip3 install scikit-learn && python3
/home/keras-on-docker.py”
```

Map the local directory at $(pwd) onto the Docker container’s file system at /home

 © 2023 General Assembly

Our Docker Command

(run from inside this lesson repo!)

```
docker run \
    –gpus all \
    -v $(pwd):/home \
    tensorflow/tensorflow:latest-gpu-py3 \
    /bin/bash -c “pip3 install scikit-learn && python3
/home/keras-on-docker.py”
```

The official TensorFlow Docker image we’re using. Almost 7GB!

 © 2023 General Assembly

Our Docker Command

(run from inside this lesson repo!)

The command we're running on the container. It's two commands combined into one:

1. Install Scikit-Learn
2. Run the given Python file



```
docker run \
    –gpus all \
    -v $(pwd):/home \
    tensorflow/tensorflow:latest-gpu-py3 \
    /bin/bash -c “pip3 install scikit-learn && python3
/home/keras-on-docker.py”
```

Conclusion

- Docker is used by many different kinds of programmers. But for us, it solves two problems:
  - Portability (“it works on my machine!”)
  - Getting Keras to recognize your GPU(s)
- There’s *much* more to Docker and its companion tools (Swarm, Kubernetes, etc), but what we’ve learned today goes a long way